

Διαχείριση Σύνθετων Δεδομένων

2η εργασία :

ΑΜ 4940 ΦΟΥΡΚΙΩΤΗΣ ΑΘΑΝΑΣΙΟΣ

Μέρος 1 (προγραμματιστικό)

- Ποιο πρόβλημα έχουμε; (1)
- Έχεις ένα αρχείο με 51.970 γραμμές. Κάθε γραμμή είναι ένα εστιατήριο στο Πεκίνο
 - Δύο αριθμοί = η θέση στον χάρτη
 - Ποια εστιατήρια υπάρχουν σε αυτήν τη διεύθυνση;
 - Τι κάνεις χωρίς ερωτήριο
 - Κοιτάς εστιατήριο 1 → είναι μέση; ΟΧΙ
 - Κοιτάς εστιατήριο 2 → είναι μέση; ΝΑΙ
 - Κοιτάς εστιατήριο 3 → είναι μέση; ΟΧΙ
 - ...
 - Κοιτάς εστιατήριο 51.970 → είναι μέση; ΟΧΙ
- } Πολύ φρό
- Λύση: Οργανώνω τα εστιατήρια χωρικά ώστε Ν-Χ
 - "Η διεύθυνση είναι στο Βόρειο-Δυτικό Πεκίνο → κοιτάξτε μόνο εκεί".
 - Έτσι λειτουργεί το R-tree.
 - Τι είναι το R-Tree;
- ΠΕΚΙΝΟ

Βόρειο-Δυτικό
A B

Νοτιοανατολικό
Γ Δ
- * Μεγάλο κομμάτι: ΡΙΖΑ (κάλυπτει όλο το πεκίνο).
 - Μεσαία κομμάτι: Εξισογισμένοι κόμβοι που κλύπτουν διευθύνσεις
 - Μικρά κομμάτια: Φύλλα που περιέχουν τα χωρικά εστιατήρια.
- Πως δουλεύει;
- 

②

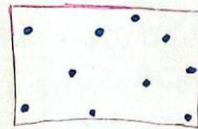
π.χ. Αν έχω εστιατήριο στο Βορειοδυτικό Πεκίνο τότε:

Ανοίγω Μεδάλιο και Βλέπω 2 μεδαλιές καινούριες

- Το Νοτιοανατολικό δεν αγγίζει τη περιοχή Άρα το αδειάζω

- Το Βορειοδυτικό το κοιτάζω και ανοίγω 2ε καινούρια Α και Β.

Στοι αυτό που πετοχέν είναι να κοιτάζω λίγα εστιατήρια και όχι και τα 51.970.



• MBR. π.χ. έχω πάλι το εστιατήριο

Βλέπω το μικρότερο ορθόγωνο που το περιέχει ΟΛΑ

$$MBR = [x_{min}, y_{min}, x_{max}, y_{max}]$$

- Κάθε κόμβος του R-tree αποθηκεύει το MBR των παιδιών του. Έτσι ξέρει τι περιοχή καλύπτει

• Τύπος Κόμβου: Τύπος 1: φύλλο. (Αποθηκεύει πρωτεύοντα εστιατήρια).

(record-id, (x, y))

↑
Αριθμός ορθογώνιου

↘
Συντεταγμένες εστιατηρίου

Τύπος 2: Ενδιάμεσος. (Αποθηκεύει δείκτες σε παιδιά + το MBR τους).

(name-id, {x-low, y-low, x-high, y-high})

↓
Παίδια

↓
το MBR των παιδιών

• Πόσα εστιατήρια χωράνε σε κάθε κόμβο;

1 εστιατήριο χρειάζεται: record-id → 4 bytes

x → 8 bytes

y → 8 bytes

20 bytes.

Άρα χωράνε $\frac{1024}{20} = 51.2 = 51$ εστιατήρια ανά φύλλο.

Για ενδιαφέρον: Πόσα παιδιά χωράνε;

1 παιδί χρειάζεται:

node-id → 4 bytes
 x-low → 8 bytes
 y-low → 8 bytes
 x-high → 8 bytes
 y-high → 8 bytes

36 Bytes.

Χωράνε: $1024 / 36 = 28.4 \approx 28$ παιδιά ανά ενδιαφέρον κόμβο.

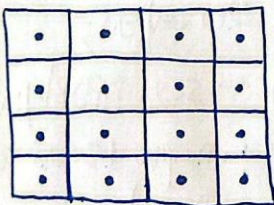
Για να βρω όμως το MIN-LEAF, MIN-INT χρησιμοποιώ τον κανόνα του 40%. Συνολικά κανόνες κόμβος ή/και της σειράς δεν μπορεί να είναι λιγότερο από 40% γεμάτος.

pro MIN-LEAF = $0.4 \cdot 51 = 20$

MIN-INT = $0.4 \cdot 28 = 11$

↑ υπάρχει πάντα 1 φύλλο
 άρα δεν σπαταλάει χώρο

• STR: bulk loading: Ξέρω εκ των προτέρων όλα τα εγχειρίδια. Τα οργανώνω εξοντα και χτίζω το δέντρο από κάτω προς τα πάνω. Αναδέιτω αν το έκανα σωστά και έβγα ένα-ένα εγχειρίδιο κάθε φορά το δέντρο θα υποβιβάζε που να το βάλει. Έτσι, το δέντρο θα εξαρτιώταν από την σειρά εισόδου και θα ήταν βέβαιο χωρίς ορί: - λειτουργική δομή.



← Κάθε τετραγωνάκι = 1 φύλλο με 51 εγχειρίδια

- 1) Ταξινομή Αριθμοί → Σειρά (κατά x)
- 2) Κόβω κάθετες Λεπίδες
- 3) Μέσω σε κάθε λείδα, Ταξινομή Κατά → Πάνω (κατά y)
- 4) Κόβω τις οριζόντιες των 51 → Κάθε οριζόντιο = 1 φύλλο
- 5) Ενοιαίαση για να φτιάξω τους δονείες



Παρατηρήσεις:

(4)

- 51.970 ενοχώρη, MAX.LEAF = 51.

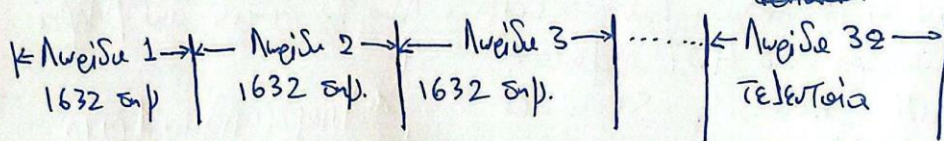
$$51.970 / 51 = 1018.6 \approx 1020 \text{ φύλλα. χρειάζεται.}$$

- $\sqrt{1020} = 31.9 \approx 32$ λωρίδες χρειάζεται

- ΠΑΡΑΤΗΡΗΣΗ: χρησιμοποιώ ρίγα δια τα "πλακέκια" διότι να είναι τετράγωνα

- Τεξινομήσει κατά x

$$32 \times 51 = 1632 \text{ σφύρα ανά λωρίδα.}$$



- Μείνω σε κάθε λωρίδα τεξινομήσει κατά y + κοπή σε ομάδες 51.

Λωρίδα 1: Σφύρα 1-51 → φύλλο 0 (Ομάδα 1)
Σφύρα 52-102 → φύλλο 1 (Ομάδα 2)

Σφύρα 1582-1632 → φύλλο 31 (Ομάδα 32)

- Έλεγχος αν η τελευταία ομάδα έχει λιγότερα από 20 σφύρα;

Πριν: [— 51 —] [— 51 —] [— 5 —] ← 5 < 20 ΠΡΟΒΛΗΜΑ
Μετά: [— 51 —] [— 36 —] [— 20 —] ← παίρνω 15 από το προτελευταίο

Επανεξέταση για καλύτερα επίπεδα.

$$1020 \text{ φύλλα} \rightarrow 1020/28 = 37, \sqrt{37} = 7 \rightarrow 37 \text{ ενδιάμεσοι (level 1)}$$

$$37 \text{ κόμβοι} \rightarrow 37/28 = 2, \sqrt{2} = 2 \rightarrow 2 \text{ ενδιάμεσοι (level 2)}$$

$$2 \text{ κόμβοι} \rightarrow 2/28 = 1 \rightarrow 1 \text{ ΡΙΖΑ (level 3)}$$

(5)

Δηλαδή level 0:

$[Φ0] [Φ1] [Φ2] [Φ3] \dots [Φ1019]$

1020 φύλλα, το καθένα με 51 εσχατόρια.

Αρα εστιά τα 1020 φύλλα τα αντιστοιχίζω σαν σελίδα.

Πρι είχα: 51.970 εσχατόρια $\rightarrow$ ΟΜΑΔΟΠΟΙΗΣΗ $\rightarrow$ 1020 φύλλα

1020 φύλλα $\rightarrow$ ΟΜΑΔΟΠΟΙΗΣΗ $\rightarrow$ 1.000.000 κόμβοι (level 1)

level 1:

1020 φύλλα $\rightarrow 1020/28 = 36.4 = 37$ κόμβους level 1.

MAX_INT = 28

$\sqrt{37} = 7$ κλάδοι δένδρου.

*Με MAX_INT = 28 δηλαδή 28

παιδιά μέγιστο χωράει κάθε κόμβος

Ταξινομώ τα 1020 φύλλα κατά κέντρο x των MBR των, κόβω σε 7 δένδρα, σε κάθε δένδρο ταξινομώ κατά κέντρο y, κόβω σε ομάδες των 28.

Αρα παραπαράκω:

Level 1: $[K0] [K1] [K2] [K3] \dots [K36]$

37 κόμβοι με τα καθένα να μπορεί να πάρει μέχρι 28 παιδιά

level 2:

$37 \text{ κόμβοι} / 28 = 1.32 \approx 2 \leftarrow 2 \text{ κόμβους χρειάζονται}$

$\sqrt{2} \approx 2$ κλάδοι δένδρου

$[K1057] [K1058]$

2 κόμβοι, ο καθένας βλέπει 19 κόμβους level 1

level 3: P12A.

Έχω 2 κόμβους: $2/28 = 1 \leftarrow$ χρειάζεται 1 κόμβος = P12A.

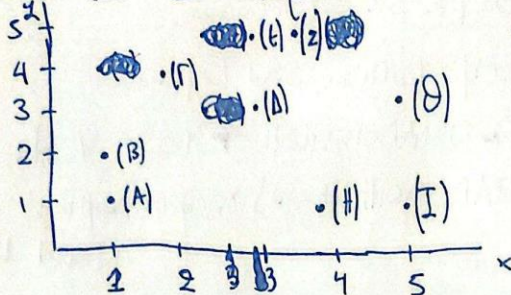
$[P12A]$: 1 κόμβος που βλέπει τα 2 κόμβους level 2.



Παράδειγμα:

Έχω 9 εδατώρια και $MAX_LEAF=3$

Έστω τα εδατώρια:



- Δέλω να τα Βάλω σε 3 φύλλα
($9:3=3$) το καθένα με 3 εδατώρια.

Αν τα έβαλε τυχόν π.χ. φύλλο 0: Α, Ζ, Θ
φύλλο 1: Β, Δ, Η
φύλλο 2: Γ, Ε, Ι } *Σκοπίζουν πάντα!*

Έτσι το ΜΒR κάθε φύλλου τρώει το δέντρο γιατί τα σημεία είναι μετρία μετ'αυτού τους δένδρα: κακό.

Άρα:

Έχω 9 εδατώρια με $MAX_LEAF=3$.

Αν δέλω 3 φύλλα και $\sqrt{3} \approx 2$ Άρα 2 κότες λωρίδες

* Χρησιμοποιώ π.χ. δένδρα δέλω τα φύλλα να καλύπτουν τετραγωνικές περιοχές στο χάρτη

Μετά κάνω Ταξινόμηση κατά x

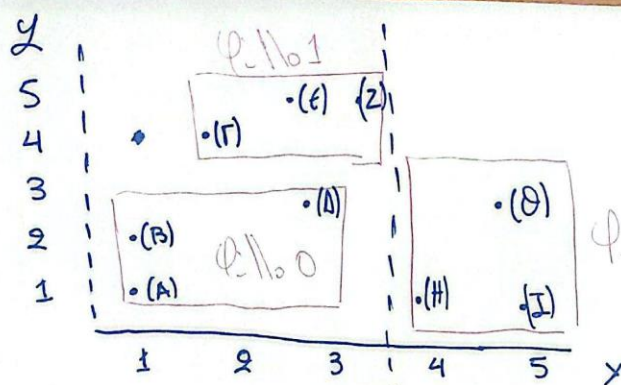
Πρω: Ε, Β, Θ, Α, Ζ, Δ, Γ, Η, Ι (Τυχόν σειρά).

Μετά Α, Β, Γ, Δ, Ε, Ζ, Η, Θ, Ι (Κατά x)
1 1 2 3 3 4 4 5 5 (x τιμές)

Μετά κοπή σε λωρίδες:

Κάθε λωρίδα πρέπει να έχει $\sqrt{3} \times MAX_LEAF = 2 \times 3 = 6$ εδατώρια

Έχω 9 εδατώρια και κάθε λωρίδα έχει μέχρι 6 εδατώρια.



(7)
 * Κάθε Φ_{No} έχει
 ήχο UPR διατί το
 εδάφιο ειχε
 κατ'επεταξί τας.

Κάθε τόπος
 Λεϊδα 1 | Λεϊδα 2.

Ταξινομήση κατω y πέδε σε κάθε Λεϊδα.

Λεϊδα 1: Έχει A, B, Γ, Δ, Ε, Ζ.

$A(y=1)$ $B(y=2)$ $\Gamma(y=4)$ $\Delta(y=3)$ $E(y=5)$ $Z(y=5)$.

Μετα τω ταξινομήση:

$A(1)$, $B(2)$, $\Delta(3)$, $\Gamma(4)$, $E(5)$, $Z(5)$.

Λεϊδα 2: Έχει: H, Θ, I

$H(y=1)$, $I(y=1)$ $\Theta(y=3)$

$H(1)$, $I(1)$, $\Theta(3)$ Μετα τω ταξινομήση.

Λεϊδα 1: 6 εδαφια : 6 : 3 ~~(ανά Φ. No)~~ (ανά Φ_{No}) = 2 Φ_{No} .

$\Phi_{No} 0$: A, B, Δ (τα 3 πιο "κάτω" τω Λεϊδας)

$\Phi_{No} 1$: Γ, Ε, Ζ (τα 3 πιο "πάνω" τω Λεϊδας)

Λεϊδα 2: 3 εδαφια : 3 (ανά Φ_{No}) = 1 Φ_{No} .

$\Phi_{No} 2$: H, I, Θ.

Μέρος 2 (προγραμματιστικό)

Ερωτήσεις Εύρους με τη χρήση Παραθύρου (Window Range Queries)

Το ερώτημα αυτό πρακτικά με ρωτάει "Πες μου όλα τα εγχα-
τόρια που είναι μέσα σε ένα συγκεκριμένο ορθογώνιο του χάρτη" 8
Το ορθογώνιο είναι το παράθυρο W με $W = [x-low, y-low, x-high, y-high]$
οπότε δέλω να βρω όλα τα σημεία του dataset που είναι μέσα στο
 W μαζί με αυτών στο ηφίδραμμα.

2 τρόποι:

αποτελεσματα = []

For κάθε σημείο στο dataset:

if σημείο μέσα στο W :

αποτελεσματα.append(σημείο)

Από! εφω
51970 ελέγχω
με 1000 queries
→ 51.970.000
ελέγχω

Με R-tree αναπτύσσεται να κλαδεύω
ολόκληρα υποδέντρα χωρίς περετέρω έλεγχο.

R-tree.

Αν θα ελέγγω αν το LBR κάθε ηφίδας τέρνεται με το
παράθυρο W → Αν ΟΧΙ: όλα τα σημεία μέσα σε αυτό το ηφίδι
είναι σίγουρα έξω από το W και δεν θα ελέγγω
καθόλου.

→ Αν ΝΑΙ: κάποια σημεία μπορεί να είναι μέσα
και να κατέβω και να ψάξω περετέρω.

• Πως όμως 2 ορθογώνια τέρνονται;

Με Απονο ισχύει: "2 ορθογώνια δεν τέρνονται αν και μόνο αν
το ένα είναι τελείως αλλοι σε μία από τις 4 διευθύνσεις
από το άλλο → αριστερά, δεξιά, πάνω, κάτω

Δηλαδή όλες οι συνθήκες των ενοίων είναι τελείως αλληλο
αποκλειόμενες με το άλλο.

(9)

Αυτό δίνεται με τους ελέγχους:

$\left\{ \begin{array}{l} A.\text{right} < B.\text{left} \longrightarrow A \text{ τελείως αριστερά από } B \\ A.\text{left} > B.\text{right} \longrightarrow A \text{ τελείως δεξιά από } B \\ A.\text{top} < B.\text{bottom} \longrightarrow A \text{ τελείως κάτω από } B \\ A.\text{bottom} > B.\text{top} \longrightarrow A \text{ τελείως πάνω από } B. \end{array} \right.$

Αν δεν ισχύει καμία από αυτές τις 4 συνθήκες τότε τέμνονται.

• $W = [x\text{-low}, y\text{-low}, x\text{-high}, y\text{-high}]$

σημείο = (x, y)

Είναι μέσα το σημείο στο ορθογώνιο αν:

$x\text{-low} \leq x \leq x\text{-high}$ ΚΑΙ $y\text{-low} \leq y \leq y\text{-high}$.

• ΦΕΥΔΟΚΟΝΙΚΑΣ:

Function window-query(W):

αποτελεσματα = $\{\}$

Βάζει τω πύρα στην στοίβα

όσο η στοίβα δεν είναι άδεια:

Πάρε ένα κόμβο από τω στοίβα

αν είναι φύλλο:

για κάθε σημείο στο φύλλο:

αν το σημείο μέσα στο W :

προσθέσε το στα αποτεσματα

αν είναι εσωτερικός:

για κάθε παιδί:

αν το MBR του παιδιού τέμνεται με το W

βάλε το παιδί στην στοίβα

return αποτελεσματα.

Ερωτήσεις Εύρους με βάση την Απόσταση (Distance Range Queries)

(10)

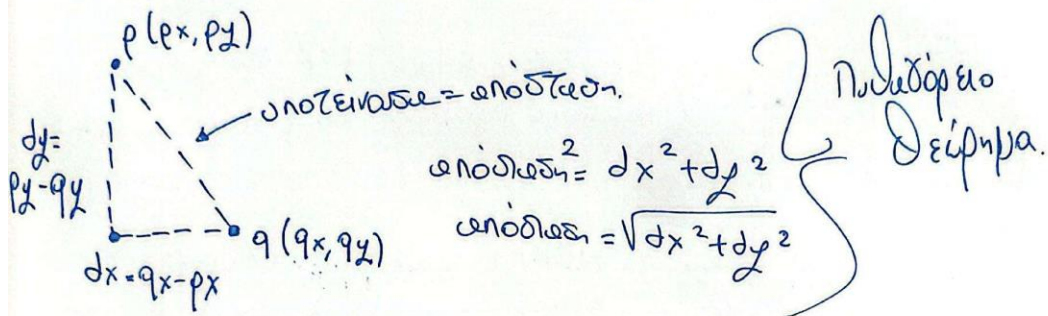
Στο πρώτο αυτό έχω να δώσω ένα πρόβλημα της τάξης:

"Δείξε μου όλα τα εστιατόρια που είναι σε απόσταση ≤ 1000 μέτρων από το αϊρόντ μου".

"Αϊρόντ μου" $\rightarrow q = (q_x, q_y)$

• Ακτίνα r που είναι η απόσταση έως 1000 μέτρα.

Οπότε έχω δέλω: απόσταση $(q, p) \leq r$.



2 Τρόποι:

αποτελεσμα = []

for κάθε σημείο (px, py) :

αποσταση = $\sqrt{(px - qx)^2 + (py - qy)^2}$

if αποσταση $\leq r$:

αποτελεσμα.append(σημείο)

Στο ελάχιστο
Από!!

Rtree:

Θυμάσαι ότι είπαμε ότι η ελάχιστη απόσταση του q από το MBR είναι μεγαλύτερη από r τότε κανένα σημείο μέσα στο MBR δεν μπορεί να είναι καλύτερο από αυτό. Άρα κάνω ότι και πριν. Σημειώστε.

• Πως βρίσκω $\text{mindist}(q, \text{MBR})$;

(11)

Εάν έχω να βρω το πιο κοντινό σημείο που είναι μέσα σε ένα MBR από το q , Θα βρω το πιο κοντινό σημείο πάνω στο MBR ως προς το q .

ΠΕΡΙΠΤΩΣΕΙΣ:

1)  } Αν q μέσα στο MBR τότε $\text{mindist}(q, \text{MBR}) = 0$

2)  } Πιο κοντινό σημείο MBR στο q είναι κάποια προβολή του q πάνω στο MBR.

Για x :

- Αν $q_x < x_{\text{low}}$ (q αριστερά του MBR) η προβολή $c_x = x_{\text{low}}$
- Αν $q_x > x_{\text{high}}$ (q δεξιά του MBR) η προβολή $c_x = x_{\text{high}}$.
- Αλλιώς q ανάμεσα δεξιά η προβολή $c_x = q_x$.

Για y :

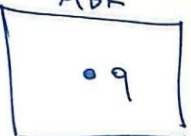
- Αν $q_y < y_{\text{low}}$... $c_y = y_{\text{low}}$ (Κάτω του MBR)
- Αν $q_y > y_{\text{high}}$... $c_y = y_{\text{high}}$ (Πάνω του MBR)
- Αλλιώς $c_y = q_y$

Παρεδείγματα:

1)  * Το q είναι πάλι αριστερά του MBR.
 $q_x < x_{\text{low}} \rightarrow c_x = x_{\text{low}}$
 $q_y > y_{\text{high}} \rightarrow c_y = y_{\text{high}}$ } $\rightarrow$ Από πιο κοντινό σημείο η πάνω-αριστερή γωνία του MBR.

2)  * Το q είναι αριστερά του MBR.
 $q_x < x_{\text{low}} \rightarrow c_x = x_{\text{low}}$
 $y_{\text{low}} \leq q_y \leq y_{\text{high}} \rightarrow c_y = q_y$

Από πιο κοντινό σημείο ~~σημείο~~ θα είναι στο φασίς ή/και το MBR (12)

3)  * $x_{low} \leq q_x \leq x_{high} \rightarrow cx = qx$
 $y_{low} \leq q_y \leq y_{high} \rightarrow cy = qy$

Από πιο κοντινό σημείο το ίδιο το Q με απόσταση = 0.

Οπότε τι πετσοκάνω; Αν $mindist > r$ κανένα σημείο δε μπορεί να έχει απόσταση $\leq r$. Και αβάνω το υποδέντρο.

• ΨΕΥΔΟΚΟΔΙΚΑΣ:

Function distance_query(q, r):

αποτελεσμα = {}

Βάζω τω είχα στοίβα

όσο η στοίβα δεν είναι άδεια:

Πάρε ένα κόμβο από τω στοίβα

α) είναι φύλλο:

για κάθε σημείο:

υπολόγισε η ελάχιστη απόσταση από το q

α) $\leq r$:

προσέθεσε το στο αποτελεσμα.

α) είναι εσωτερικός:

για κάθε παιδί:

α) $mindist(q, child_MBR) \leq r$:

κάνε το child push στοίβα

return αποτελεσμα.

Ερωτήσεις k Πλησιέστερων Γειτόνων (kNN Queries)

Στο πρώτο αυτό που ζητάται η x

(13)

"Ποιόν και δέλω να που δείξεις τα 10 πιο κοντινά ενοχοποιημένα από ΕΜΕΝΑ."

Οπότε έχω το k που εδώ είναι 10 (αριθμός ενοχοποιήσεων)
και το σημείο αναφοράς $q = (q_x, q_y)$

Αρα δέλω-ψάχνω τα k σημεία του dataset με τη μικρότερη απόσταση από το q. Όπως να είναι ταξινομημένα από το πιο κοντινό προς το πιο μακρινό.

2 Τρόποι:

Apostuseis = []

For κάθε σημείο:

$$d = \text{sqrt}((x - q_x)^2 + (y - q_y)^2)$$

~~Distance~~

Apostuseis.append((d, σημείο))

~~Sort~~

Apostuseis.sort()

return Apostuseis[:k]

Υπολογίζω απόσταση
από όλα τα 51970
σημεία και μετά
τα ταξινομώ
Από!!

Best-First Search:

Αντί να κάνω είναι να εστιάζω τα > κόμβους που είναι πιο κοντά στο q. Οπότε πόλλες βρω k σημεία να έχω βρει τα k πιο κοντινά και δεν χρειάζεται να κοιτάξω τίποτα άλλο

Αντί να σωλεύω με priority Queue (Ουρά προτεραιότητας)

Η Priority Queue "βγάζει" πάντα το πιο σημαντικό στοιχείο

Χωρίς να το νοιάζει πότε μπήκε. Το σημαντικό στοιχείο (14)
στο priority queue αυτό = μικρότερη Απόσταση από το q.
Αντίθετος για Queue ακολουθεί FIFO και η στίβος (stack)
ακολουθεί LIFO που δεν με εξυπηρετών στο priority queue
αυτό και για αυτό χρησιμοποιούμε το Priority Queue.

• ΨΕΥΔΟΚΩΔΙΚΑΣ:

Function knn-query (q, k)

αποτελεσμα = {}

heap = {}

Βάζει στο heap: (0.0, "N", rize_id)

όσο heap δεν είναι άδειο και len(αποτελεσμα) < k:

Πάρε από το heap το στοιχείο με τη μικρότερη απόστα-
ση

αν είναι σημείο (ετικέτα "P"):

προσθήκη στα αποτελεσμα

αν είναι κόμβος (ετικέτα "N"):

αν είναι φύλλο:

για κάθε σημείο γείτονα:

υπολόγισε ημιδιαστάση απόσταση

push στο heap με ετικέτα "P"

αν είναι ενδιαμέσος:

για κάθε γείτονα:

υπολόγισε mindist(q, child-MBR)

push στο heap με ετικέτα "N"

return αποτελεσμα.

(15)

Το υλοποιώ με heap αφαίρεση "ταβίξω" το στοιχείο με την
πικρότερη απόσταση. Έστω σημείο A με απόσταση d τότε όλα τα
υπόλοιπα στοιχεία στο heap έχουν απόσταση $\geq d$. Άρα λογικό είναι
αυτό το σημείο να είναι σίγουρα ο επόμενος πιο κοντίνος δείκτης.
Αυτό συμβαίνει αφα στο heap τα κανονικά σημεία έχουν την
πρόσθετική τους απόσταση. Αν ισχύει $\geq d$, είναι πεφτισμένα.
Επίσης στο heap παρατηρούμε ότι όλοι οι κόμβοι έχουν $\text{mindist} \geq d$.
Άρα τα εσωτερικά τους σημεία μπορούν να είναι $\text{max} \geq \text{mindist} \geq d$.
Αντίστοιχα κανένα σημείο που δεν έχω βάλει ακόμα δεν είναι πιο
κοντά. Έτσι αποδεικνύεται ότι κάθε σημείο που βάλω από
το heap είναι ο επόμενος KNN με σωστή σειρά.